PRACTICAL DEVELOPER ASSESSMENT

Secure Asynchronous Employee REST API

Technology track: Spring Boot / Python FastAPI / ASP.NET Core Web API | Duration: 120 minutes | Total: 100 marks

1. Assessment objective

Develop a secure REST API that authenticates an API client using Client ID and Client Secret, issues an access token, initializes an employee submission transaction, accepts validated employee data, processes the insertion asynchronously, and allows the authenticated client to retrieve transaction status and the saved employee record using the transaction ID.

All technology tracks must follow the same API contract, validation rules, status lifecycle, security requirements, and acceptance tests. A preconfigured project skeleton and database connection may be supplied.

2. Employee data and validation rules

Field	Type	Required validation
name	String	Required; trimmed; 2–100 characters. Allow letters, spaces, periods, apostrophes and hyphens. Reject blank or whitespace-only values.
dob	Date	Required; ISO format YYYY-MM-DD; must be a valid calendar date in the past.
gender	Enum/String	Required; allowed values exactly MALE, FEMALE, OTHER (case normalization may be implemented consistently).
pan	String	Required; uppercase Indian PAN pattern: 5 letters + 4 digits + 1 letter (for example, ABCDE1234F). Store as text.
mobile_number	String	Required; exactly 10 digits; first digit must be 6, 7, 8 or 9. Store as text.

Database rules: PAN must be unique. Mobile number should be stored as a string; uniqueness is optional unless the assessment environment specifies otherwise. Validate on the server regardless of any client-side checks.

3. Required API contract

Method and endpoint	Purpose / expected behavior
POST /api/v1/auth/token	Accept client_id and client_secret. Validate credentials and return a short-lived bearer access token. Never store or log plaintext secrets.
POST /api/v1/transactions/init	Requires Bearer token. Create a unique transaction associated with the authenticated client. Return transaction_id and INITIALIZED status.
POST /api/v1/transactions/{transactionId}/employees	Requires Bearer token. Validate transaction ownership/state and employee payload. Queue or schedule asynchronous processing. Return HTTP 202 Accepted.
GET /api/v1/transactions/{transactionId}	Requires Bearer token. Return transaction status and safe error details if processing failed.
GET /api/v1/transactions/{transactionId}/employees	Requires Bearer token. Return employee data only after successful database commit; enforce client ownership.

4. Sample requests and responses

4.1 Token request

POST /api/v1/auth/token

Request JSON: {"client_id":"assessment_client","client_secret":"provided-secret"}

Response example: {"access_token":"<token>","token_type":"Bearer","expires_in":1800}

4.2 Initialize transaction

POST /api/v1/transactions/init | Authorization: Bearer <token>

Response example: {"transaction_id":"TXN-8F21A9C4","status":"INITIALIZED","message":"Transaction initialized successfully"}

4.3 Submit employee data

POST /api/v1/transactions/TXN-8F21A9C4/employees | Authorization: Bearer <token>

Request JSON:

```
{
  "name": "Amit Kumar",
  "dob": "1990-05-15",
  "gender": "MALE",
  "pan": "ABCDE1234F",
  "mobile_number": "9876543210"
}
```

Acknowledgement: HTTP 202 Accepted — {"transaction_id":"TXN-8F21A9C4","status":"ACCEPTED","message":"Employee data accepted for processing"}

4.4 Check status and retrieve saved record

GET /api/v1/transactions/TXN-8F21A9C4

In progress: {"transaction_id":"TXN-8F21A9C4","status":"PROCESSING"}

Completed: {"transaction_id":"TXN-8F21A9C4","status":"COMPLETED","message":"Employee data saved successfully"}

GET /api/v1/transactions/TXN-8F21A9C4/employees

Return the employee record read from the database, together with transaction_id and status. Do not simply echo the submitted payload.

5. Transaction lifecycle and asynchronous processing

Status	Meaning
INITIALIZED	Transaction created; waiting for employee payload.
ACCEPTED	Payload accepted and processing scheduled; response returned without waiting for persistence.
PROCESSING	Background worker is validating/persisting the record.
COMPLETED	Database transaction committed and saved data is available for retrieval.
FAILED	Processing failed; safe failure reason is recorded and no partial employee record remains.

Use a framework-supported background task, executor, hosted service, or queue. A separate message broker is not mandatory. The implementation must not report COMPLETED until the database commit succeeds. Ensure employee insertion and final transaction status are consistent.

6. Security and implementation requirements

- Use a standard client-credentials pattern for token issuance; access tokens must expire.
- Validate the bearer token and required scope/permission on every protected endpoint.
- Bind every transaction to the authenticated client and prevent cross-client access, including guessed transaction IDs.
- Use parameterized ORM/database operations; do not concatenate user input into SQL.
- Return consistent validation errors and appropriate HTTP status codes; never expose stack traces or secrets.

- Prevent duplicate employee records through a unique PAN constraint and handle duplicate submissions/idempotency safely.
- Record transaction timestamps, processing status and safe failure reasons. Avoid logging full PAN, mobile number, client secret or access token.
- Handle concurrent requests and database failures without inconsistent status or partial employee records.

7. HTTP status expectations

HTTP status	Use
200 OK	Successful status or saved-data retrieval.
201 Created	Optional for synchronous transaction creation if the candidate documents the choice; 200 is also acceptable for init.
202 Accepted	Employee submission accepted for asynchronous processing.
400 Bad Request	Malformed JSON or field validation failure.
401 Unauthorized	Missing, invalid or expired token; invalid client credentials.
403 Forbidden	Authenticated client lacks permission, where distinguished from ownership concealment.
404 Not Found	Unknown transaction or resource; may also be used to avoid revealing another client's resource.
409 Conflict	Duplicate PAN, invalid transaction state, or conflicting repeated submission.
500 Internal Server Error	Unexpected failure with no sensitive details exposed.

8. Time allocation and marks

Assessment component	Time	Marks
Authentication and authorization	25 minutes	20
Transaction initialization and employee persistence	30 minutes	25
Asynchronous processing and status tracking	25 minutes	25
Retrieval, validation and error handling	25 minutes	20
Code quality and demonstration	15 minutes	10
TOTAL	120 minutes	100

9. Evaluator acceptance tests

1. Valid client credentials produce a token; invalid credentials are rejected.
2. Missing, malformed, invalid and expired tokens cannot call protected APIs.
3. A valid token initializes a unique transaction tied to that client.
4. Valid employee payload is accepted with HTTP 202 and a transaction ID.
5. Invalid name, future/invalid DOB, unsupported gender, malformed PAN and invalid mobile number are rejected.
6. Unknown transaction IDs and transactions owned by another client cannot be used to submit or retrieve data.
7. Transaction status progresses through appropriate states and reaches COMPLETED only after database commit.
8. Saved employee data is read back from the database and matches the submitted valid data.
9. Duplicate PAN and repeated submissions are handled without creating duplicate records.
10. A simulated persistence failure results in FAILED status and no partial employee record.
11. Concurrent submissions do not corrupt transaction state or bypass uniqueness constraints.
12. Errors are consistent and do not expose secrets, stack traces or unnecessary personal data.

10. Scoring rubric

Competency	Marks	What earns full marks
Client authentication and token security	20	Credential verification, token expiry, protected endpoints and authorization checks.
Transaction API and data model	15	Unique transaction ID, client ownership, appropriate status/state model.
Employee validation and persistence	20	All five fields validated server-side; correct database constraints and ORM/persistence.
Asynchronous processing	20	Non-blocking acceptance, background processing, accurate state transitions and failure handling.
Status/data retrieval and isolation	10	Correct retrieval from database; ownership enforced.
Error handling and idempotency	5	Consistent HTTP responses, duplicate/retry handling and safe errors.

Code quality, tests and demonstration	10	Readable structure, separation of concerns, meaningful tests and working demonstration.
TOTAL	100	

11. Candidate submission checklist

- ☐ Working source code in the assigned technology.
- ☐ README with setup/run instructions and any assumptions.
- ☐ Database schema/migration or entity definitions.
- ☐ API test collection or automated tests covering success and failure cases.
- ☐ Brief demonstration of token generation, transaction initialization, asynchronous insertion and retrieval.

12. Candidate details and evaluator score sheet

Candidate name / ID	Technology	Evaluator	Date

Competency	Maximum	Score awarded	Evaluator remarks
Authentication and authorization	20		
Transaction API and data model	15		
Validation and persistence	20		
Asynchronous processing	20		
Retrieval and isolation	10		
Error handling and idempotency	5		
Code quality and demonstration	10		
TOTAL	100		

End of Assessment Document